FoodLoop System — UML Class Diagram

Step-by-Step Guide for StarUML & Visual Paradigm

📋 Overview of Classes Identified

Class	Type	Source
`User`	Base class (with inheritance)	`userSchema` in server.js
`Admin` / `NGO`	Subclasses of User	Role-based logic in auth
`Listing`	Entity	`listingSchema`
`Request`	Entity	`requestSchema`
`NgoRequest`	Entity	`ngoRequestSchema`
`Pickup`	Entity	`pickupSchema`
`Partner`	Entity	`partnerSchema`
`Waste`	Entity	`wasteSchema`
`Activity`	Entity	`activitySchema`
`Metric`	Entity	`metricSchema`

🔧 Option A: StarUML (Step-by-Step)

Step 1: Create New Project

1. Open **StarUML**
2. `File → New → Empty Project`
3. Save as `FoodLoop.mdj`

Step 2: Create Class Diagram

1. Right-click project → `Add Diagram → Class Diagram`
2. Name it: `FoodLoop Class Diagram`

Step 3: Add the `User` Base Class

1. Drag **Class** from Toolbox to canvas
2. Name: `User`
3. Click class → `Properties` panel → Add attributes:

Attribute	Type	Visibility	Notes
`name`	String	+	required
`email`	String	+	required, unique
`password`	String	#	hashed (protected)
`role`	Enum {admin, ngo}	+	required
`createdAt`	Date	+	default: now
`_id`	ObjectId	+	MongoDB ID

4. Add operations:

Operation	Return	Visibility
`signup(name, email, password, role)`	User	+
`signin(email, password, role)`	User	+
`verify(token)`	User	+
`logout()`	void	+

Step 4: Create Subclasses (Inheritance)

1. Drag **Generalization** (solid line with hollow triangle) from Toolbox
2. Create two child classes: `Admin` and `NGO`
3. Connect: `Admin` → `User`, `NGO` → `User`
4. No additional attributes needed (role differentiates behavior)

Step 5: Add `Listing` Class

Attribute	Type	Visibility	Default/Notes
`emoji`	String	+	'🍽'
`name`	String	+	required
`detail`	String	+	optional
`quantity`	String	+	required (e.g., "35 boxes")
`expiry`	String	+	required (e.g., "Today, 8:00 PM")
`image`	String	+	URL from TheMealDB
`status`	Enum {Available, Reserved}	+	default: Available
`requests`	String	+	default: "0 requests"
`createdAt`	Date	+	default: now
`_id`	ObjectId	+	MongoDB ID

Operations:

- `create(data)` → Listing
- `resolveFoodImage(name)` → String (URL)

Step 6: Add `Request` Class

Attribute	Type	Visibility	Notes
initials	String	+	e.g., "HF"
color	String	+	UI color: blue, green, purple, etc.
name	String	+	NGO/partner name
item	String	+	requested food item
time	String	+	"Received just now"
status	Enum {pending, accepted, declined}	+	default: pending
createdAt	Date	+	default: now
_id	ObjectId	+	MongoDB ID

Operations:

- `accept()` → void
- `decline()` → void

Step 7: Add `NgoRequest` Class

Attribute	Type	Visibility	
foodName	String	+	
ngoName	String	+	default: "NGO"
createdAt	Date	+	
_id	ObjectId	+	

Step 8: Add `Pickup` Class

Attribute	Type	Visibility	Notes
day	Enum {Today, Tomorrow, Completed}	+	default: Today
time	String	+	e.g., "4:30 PM"
item	String	+	required
partner	String	+	optional
_id	ObjectId	+	

Step 9: Add `Partner` Class

Attribute	Type	Visibility	Notes
initials	String	+	2-letter code
name	String	+	required
detail	String	+	e.g., "NGO · 8.4 km away"
color	String	+	default: blue
status	String	+	default: "Connected"
_id	ObjectId	+	

Step 10: Add `Waste` Class

Attribute	Type	Visibility	Notes
date	String	+	formatted date
category	String	+	e.g., "Prepared meals"
quantity	String	+	e.g., "8 kg"
reason	Enum {Overproduction, Spoilage, Quality issue}	+	
loggedBy	String	+	default: "Admin"
createdAt	Date	+	
_id	ObjectId	+	

Step 11: Add `Activity` Class

Attribute	Type	Visibility	
icon	String	+	
iconClass	String	+	CSS class: green-bg, blue-bg, etc.
title	String	+	
description	String	+	
time	String	+	
createdAt	Date	+	
_id	ObjectId	+	

Step 12: Add `Metric` Class

Attribute	Type	Visibility	
key	String	+	default: "dashboard"
foodDiverted	Number	+	kg
mealsShared	Number	+	
pendingPickups	Number	+	
activePartners	Number	+	
diversionPct	String	+	
mealsPct	String	+	
mealsGoal	Number	+	
mealsGoalPct	Number	+	
updatedAt	Date	+	
_id	ObjectId	+	

🔗 Step 13: Add Associations (Relationships)

Association 1: User → Listing (1-to-Many)

- **User** "publishes" **Listing**
- Multiplicity: `1` → `*`
- Role name: `provider`

Association 2: NGO → Request (1-to-Many)

- **NGO** "sends" **Request**
- Multiplicity: `1` → `*`
- Role name: `requester`

Association 3: Request ↔ Listing (Many-to-1)

- **Request** "references" **Listing**
- Multiplicity: `*` → `1`
- Role name: `foodItem`

Association 4: NGO → NgoRequest (1-to-Many)

- **NGO** "tracks" **NgoRequest**
- Multiplicity: `1` → `*`

Association 5: Request → Activity (1-to-Many)

- When request accepted/declined → creates **Activity**
- Multiplicity: `1` → `*`

Association 6: Listing → Activity (1-to-Many)

- When listing created → creates **Activity**
- Multiplicity: `1` → `*`

Association 7: Pickup ↔ Request/Listing (Many-to-1)

- **Pickup** "fulfills" **Request** / **Listing**
- Multiplicity: `*` → `1`

Association 8: Partner ↔ Pickup (1-to-Many)

- **Partner** "performs" **Pickup**
- Multiplicity: `1` → `*`

Association 9: User → Waste (1-to-Many)

- **User** "logs" **Waste**
- Multiplicity: `1` → `*`
- Role name: `loggedBy`

Association 10: Partner Types

- **Partner** can be: NGO, Kitchen, Shelter, FoodBank
- Use **Generalization** from specific types to Partner
- Or use `partnerType()` logic in code (enum-like)

🎨 Step 14: Styling & Layout (StarUML)

1. **Select all classes** → Right-click → `Auto Layout → Hierarchical`
2. **Color code by package/layer**:
    - Auth layer (User, Admin, NGO) → Blue tint
    - Core domain (Listing, Request, NgoRequest) → Green tint
    - Operations (Pickup, Partner, Waste) → Orange tint
    - Analytics (Activity, Metric) → Purple tint
3. **Show multiplicities** on all association ends
4. **Add package** (dashed rectangle) for grouping:
    - `Authentication`
    - `Food Management`
    - `Logistics`
    - `Analytics`

🔧 Option B: Visual Paradigm (Step-by-Step)

Step 1: New Project

1. `Project → New` → Name: `FoodLoop`
2. Select **Class Diagram** as initial diagram

Step 2: Create Classes (Using Resource Catalog)

1. Click **Class** in diagram toolbar
2. Click canvas → Name: `User`
3. Press `Enter` → Add attributes via Resource Catalog (bottom-right of class)
4. Right-click attribute → `Add Attribute` (repeat for all)

Step 3: Inheritance

1. Hover `User` class → Click **Generalization** handle (triangle)
2. Drag to empty space → New class: `Admin`
3. Repeat for `NGO`

Step 4: Associations

1. Hover source class → Click **Association** handle
2. Drag to target class
3. Double-click association line → Set:
    - Source multiplicity / role name
    - Target multiplicity / role name
    - Navigability (arrow direction)

Step 5: Enumerations

1. Toolbox → **Enumeration**
2. Create: `UserRole` {admin, ngo}
3. Create: `ListingStatus` {Available, Reserved}
4. Create: `RequestStatus` {pending, accepted, declined}
5. Create: `PickupDay` {Today, Tomorrow, Completed}
6. Create: `WasteReason` {Overproduction, Spoilage, Quality issue}
7. Link to attributes via **Association** or set attribute type to Enum

Step 6: Model Explorer Organization

1. View → `Model Explorer`
2. Right-click root → `New Model → Package`
3. Create packages: `auth`, `domain`, `logistics`, `analytics`
4. Drag classes into appropriate packages

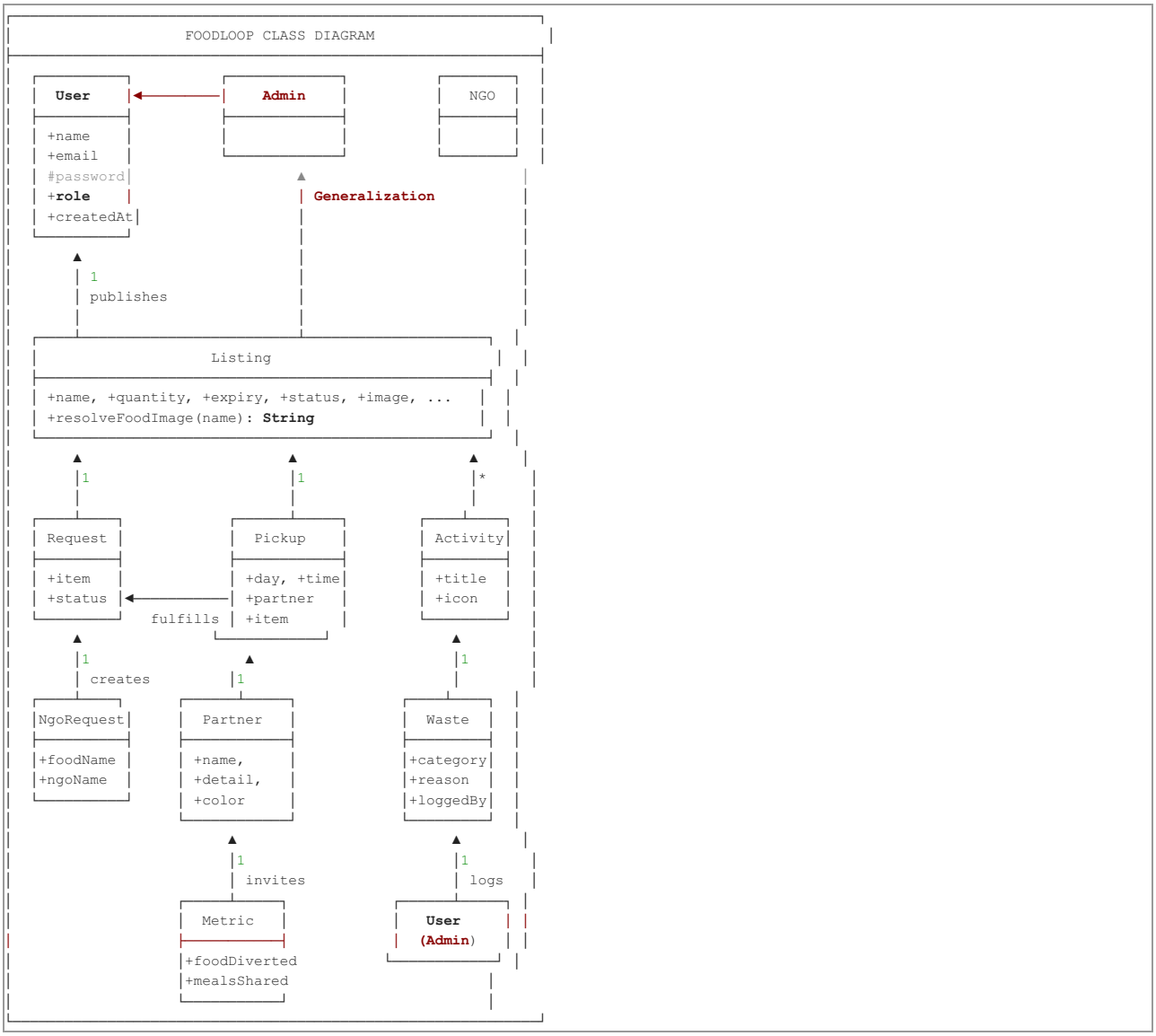
Step 7: Generate Documentation

1. `Tools → Doc. Composer → New`
2. Select `Class Diagram` → Generate HTML/PDF report

📐 Key Design Decisions to Show

Decision	Representation
Role-based auth	Inheritance: `User` ← `Admin`/`NGO`
Soft delete / status flags	Enum attributes (`status` fields)
MongoDB IDs	`_id: ObjectId` on every entity
Timestamps	`createdAt` / `updatedAt` on all
Activity logging	Separate `Activity` class linked via associations
Photo resolution	`Listing::resolveFoodImage()` operation
Partner categorization	`Partner::detail` contains type; or subclass

🖼️ Expected Diagram Structure



Validation Checklist

After drawing, verify:

- [ ] All 10 classes present
- [ ] `User` → `Admin`/`NGO` generalization shown
- [ ] All attributes match MongoDB schemas
- [ ] All associations have multiplicities (1, *, 0..1)
- [ ] Role names on association ends (provider, requester, etc.)
- [ ] Enumerations defined for status fields
- [ ] Packages group related classes
- [ ] Operations shown for key behaviors
- [ ] MongoDB `_id` and timestamps on all entities

Pro Tips

Tool	Tip
StarUML	Use `Ctrl+Shift+D` to duplicate classes; `F2` to rename
Visual Paradigm	Right-click class → `Show/Hide Attributes/Operations` for cleaner view
Both	Export as SVG for documentation: `File → Export → SVG`

Deliverable

Save as:

- `FoodLoop_Class_Diagram.mdj` (StarUML)
- `FoodLoop_Class_Diagram.vpp` (Visual Paradigm)
- Export PNG/SVG for reports

This diagram accurately reflects the actual implementation in `server.js` and `app.js` — not a theoretical design.